

WEB INFORMATION SYSTEMS

LAB ASSIGNMENT

PHP + MySQL: Database, Table Creation, and Data Insertion

Course	Web Information Systems	Lab Type	Individual practical work
Technology	PHP, MySQL, Bootstrap 5	Database API	MySQLi - Object-Oriented

1. Lab Goal

Build a small PHP application that accepts input from Bootstrap forms and uses MySQLi Object-Oriented syntax to create a MySQL database, create a table, and insert student records.

2. Learning Outcomes

- Create a MySQL connection using the mysqli class and object-oriented syntax.
- Receive form data in PHP using the POST method.
- Create a database dynamically from a user-supplied database name.
- Create a students table with appropriate columns and a primary key.
- Insert form data into the students table and display a success/error message.
- Apply basic input validation and use Bootstrap for a clean user interface.

3. Required Environment

- XAMPP or WAMP with Apache and MySQL running.
- A code editor such as VS Code.
- A browser and phpMyAdmin for verifying the database/table/records.
- Create a project folder inside htdocs, for example: htdocs/wis_lab_db/.

4. Scenario

You are developing the first part of a simple Student Information System. An administrator should be able to create the system database and then add student records through a web form. Your application will be developed in three stages.

5. Part A - Create the Database

Create a file named **create_database.php**. The page must contain one Bootstrap input field where the user enters a database name and a button labeled Create Database.

Requirements

- Use method="POST" in the form.
- Read the submitted value from \$_POST["database_name"].
- Connect to MySQL using: new mysqli("localhost", "root", "").
- Check \$conn->connect_error before executing SQL.
- Validate the database name: allow only letters, numbers, and underscores.
- Execute a CREATE DATABASE statement using \$conn->query(...).
- Show a Bootstrap alert for success or failure, then close the connection.

Starter structure - complete the missing parts

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $databaseName = trim($_POST["database_name"]);

    $conn = new mysqli("localhost", "root", "");

    // TODO 1: check connection
    // TODO 2: validate database name
    // TODO 3: build CREATE DATABASE SQL
    // TODO 4: execute query and display result
    // TODO 5: close connection
}
?>
```

Checkpoint A

- Enter a database name such as wis_lab.
- Open phpMyAdmin and verify that the database exists.
- Take a screenshot showing the successful web message and the new database in phpMyAdmin.

6. Part B - Create the students Table

Create a second file named **create_table.php**. Connect to the database created in Part A and create a table named students.

Required Table Structure

Column	Type	Length	Constraint	Purpose
id	INT	-	PRIMARY KEY, AUTO_INCREMENT	Unique student ID
full_name	VARCHAR	100	NOT NULL	Student full name
email	VARCHAR	120	NOT NULL	Student email
department	VARCHAR	80	NOT NULL	Student department
created_at	TIMESTAMP	-	DEFAULT CURRENT_TIMESTAMP	Insertion time

Connection starter

```
$conn = new mysqli("localhost", "root", "", "wis_lab");

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// TODO: write CREATE TABLE students (...) SQL
// TODO: execute it with $conn->query($sql)
// TODO: show success/error message
```

Checkpoint B

- Run create_table.php once.
- In phpMyAdmin, open wis_lab > students > Structure.
- Confirm that id is the primary key and AUTO_INCREMENT is enabled.

7. Part C - Insert Student Data from a Bootstrap Form

Create a third file named **insert_student.php**. The form must insert a new row into the students table.

Form fields

Field	HTML name	Requirement
Full Name	full_name	Required
Email	email	Required; use input type="email"
Department	department	Required

Minimum Bootstrap form structure

```
<form method="POST">
  <div class="mb-3">
    <label class="form-label">Full Name</label>
    <input type="text" name="full_name" class="form-control" required>
  </div>

  <!-- TODO: add Email field -->
  <!-- TODO: add Department field -->

  <button type="submit" class="btn btn-primary">Save Student</button>
</form>
```

PHP insertion logic - complete it

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $fullName = trim($_POST["full_name"]);
    $email = trim($_POST["email"]);
    $department = trim($_POST["department"]);

    // TODO 1: create mysqli connection to wis_lab
    // TODO 2: validate that fields are not empty
    // TODO 3: write INSERT INTO students (...) VALUES (...)
    // TODO 4: execute with $conn->query($sql)
    // TODO 5: display Bootstrap success/error alert
    // TODO 6: close connection
}
```

Required test data

Full Name	Email	Department
Ahmad Rahimi	ahmad@example.com	Information Systems

Laila Noori	laila@example.com	Computer Science
Farid Hamidi	farid@example.com	Software Engineering

8. Part D - Verification and Small Improvements

1. Insert at least three student records using your web form - do not insert them directly in phpMyAdmin.
2. Open phpMyAdmin > wis_lab > students > Browse and verify that all three records exist.
3. Try submitting the form with an empty required field and observe the browser validation.
4. Try an invalid email address and confirm that the email input provides basic validation.
5. Add a Clear/Reset button using Bootstrap class btn btn-secondary.
6. After successful insertion, clear the form or show a clear success message such as: Student added successfully.

9. Challenge Task - Recommended

Improve the insertion code by using a **prepared statement** instead of putting user values directly inside the SQL string. Use the following method names as guidance:

```
$stmt = $conn->prepare(...);
$stmt->bind_param(...);
$stmt->execute();
```

Do not copy a complete solution from another source. You should be able to explain what prepare(), bind_param(), and execute() do.

10. Expected Application Flow

1. Enter DB name



2. Create DB



3. Create students table



4. Submit student form

11. Submission Requirements

- Submit the project folder containing create_database.php, create_table.php, and insert_student.php.
- Your code must use MySQLi Object-Oriented syntax - not PDO and not procedural mysqli functions.
- Bootstrap must be used for the form, buttons, and feedback messages.
- Submit screenshots of: (1) database created, (2) students table structure, and (3) at least three inserted rows.
- Be prepared to run the application and explain the role of new mysqli(), connect_error, query(), \$_POST, and close().

12. Assessment Checklist

Criterion	Marks	Check
Database creation + correct students table	30	☐
Bootstrap student insertion form	20	☐
INSERT query, validation, and successful data insertion	30	☐
Verification, code organization, and explanation	20	☐